

# Problema de Empacotamento Unidimensional (1D-BPP): Implementação e Validação do Modelo de Kantorovich

1<sup>nd</sup> Maria Eduarda Teixeira Souza

Departamento de Computação

CEFET-MG

Divinópolis, Brasil

maria@aluno.cefetmg.br

**Abstract**—Este trabalho apresenta um estudo prático de Otimização Combinatória focado no Problema de Empacotamento Unidimensional (1D-BPP). Baseando-se na revisão de literatura conduzida por Delorme, Iori e Martello (2016) [1], foi implementada a formulação matemática de Kantorovich, referenciada na literatura como BASIC ILP, utilizando a linguagem Python e o solver Gurobi [2]. O objetivo consistiu em reproduzir os experimentos computacionais em um subconjunto de nove classes de instâncias clássicas, tais como Falkenauer, Scholl e Hard28. Os resultados obtidos corroboram as conclusões do estudo original, demonstrando as limitações computacionais do modelo em instâncias com características estruturais específicas, como *triplets*, ou que apresentam capacidades de elevada magnitude. O trabalho evidencia os desafios práticos inerentes à resolução exata deste problema.

**Index Terms**—Otimização Combinatória, Problema de Empacotamento Unidimensional, Programação Linear Inteira, BASIC ILP, Gurobi.

## I. INTRODUÇÃO AO PROBLEMA

O Problema de Empacotamento Unidimensional (1D-BPP - *One-Dimensional Bin Packing Problem*) é amplamente estudado na área de Otimização Combinatória. Formalmente, o problema recebe um conjunto de  $n$  itens, em que cada item  $j$  possui um peso  $w_j$ . Estes itens devem ser alocados em caixas idênticas, cada qual possuindo uma capacidade limite denotada por  $C$ . O objetivo fundamental consiste em empacotar todos os itens utilizando o menor número possível de caixas, respeitando a premissa de que a soma dos pesos alocados em uma dada caixa não pode ultrapassar a sua respectiva capacidade.

Do ponto de vista computacional, o 1D-BPP é classificado como um problema NP-difícil no sentido forte. Suas aplicações práticas são diversas, englobando cenários como o corte de materiais na indústria (papel, madeira e alumínio) e processos de alocação, como o armazenamento de arquivos e o carregamento de contêineres.

O objetivo deste trabalho é demonstrar a experiência prática na reprodução de estudos científicos desta área, abordando especificamente a implementação computacional de um modelo matemático clássico. Para realizar esta reprodução, o problema foi resolvido por meio de técnicas de Programação

Matemática, utilizando o *solver* Gurobi e a linguagem de programação Python.

No contexto das exigências metodológicas deste estudo — que demandam a implementação integral de uma formulação via API de um *solver* —, optou-se por focar no clássico Modelo de Kantorovich, referenciado no artigo base como *BASIC ILP*. A escolha por esta formulação, puramente baseada em Programação Linear Inteira, em detrimento de abordagens algorítmicas avançadas como o *Branch-and-Price*, deu-se de maneira estratégica. Enquanto métodos como o *Branch-and-Price* exigem a construção manual de *frameworks* complexos de geração de colunas que extrapolam o escopo de uso nativo da API, o *BASIC ILP* atende perfeitamente à proposta de modelagem matemática direta. Isso permitiu avaliar a eficácia isolada do motor de otimização na resolução exata das equações do problema.

## II. RESUMO DO ARTIGO ESCOLHIDO

O artigo selecionado, intitulado “*Bin packing and cutting stock problems: Mathematical models and exact algorithms*”, publicado por Delorme, Iori e Martello (2016) [1], constitui uma revisão ampla e sistemática (*survey*) da literatura voltada à resolução exata dos problemas de empacotamento unidimensional (*Bin Packing Problem* - BPP) e de corte de estoque (*Cutting Stock Problem* - CSP). O propósito central dos autores consiste em fornecer uma estrutura unificada das formulações matemáticas desenvolvidas ao longo de décadas e realizar uma avaliação empírica do desempenho das principais ferramentas de *software* disponíveis.

A estrutura do texto divide-se na análise de diferentes abordagens metodológicas. Inicialmente, os autores revisam o modelo clássico de atribuição binária de Kantorovich, apontando que, apesar de sua formulação compacta e simplicidade de implementação, o seu comportamento computacional prático é limitado devido à fraca relaxação linear e à redundância simétrica no espaço de busca.

No âmbito dos algoritmos exatos, o estudo examina esquemas de enumeração baseados em *Branch-and-Bound* clássicos, métodos de Programação por Restrições (*Constraint Programming*) e, de forma proeminente, algoritmos de *Branch-*

*and-Price*. O *Branch-and-Price*, fundamentado na técnica de geração de colunas de Gilmore e Gomory, destaca-se historicamente por fornecer limites inferiores (*lower bounds*) bastante ajustados para o problema.

O artigo apresenta uma avaliação experimental de doze códigos computacionais sob um ambiente de execução unificado. Os experimentos utilizaram tanto instâncias consagradas da literatura (como os conjuntos de Falkenauer e Scholl) quanto novas instâncias propostas pelos próprios autores, denominadas *Augmented Non-IRUP* (ANI), projetadas especificamente para testar os limites dos algoritmos na comprovação matemática de otimalidade.

A conclusão do estudo indica que o avanço no desenvolvimento dos *solvers* comerciais de Programação Linear Inteira permitiu que os modelos pseudo-polinomiais se tornassem altamente competitivos na atualidade, rivalizando diretamente com o *Branch-and-Price*. Contudo, o artigo ressalta que certas estruturas de dados, como problemas contendo *triplets* ou instâncias ANI com capacidades de elevada magnitude, ainda impõem severas dificuldades aos métodos exatos modernos.

### III. DESCRIÇÃO DA FORMULAÇÃO MATEMÁTICA

Para a modelagem matemática e subsequente implementação computacional, foi adotada a formulação pioneira proposta por Kantorovich, referenciada no estudo de Delorme, Iori e Martello (2016) como o modelo BASIC ILP [1]. Esta formulação representa a base clássica da Programação Linear Inteira aplicada ao Problema de Empacotamento Unidimensional (1D-BPP).

A modelagem do problema é construída a partir da seguinte definição de parâmetros e variáveis de decisão:

#### A. Parâmetros

- $n$ : número total de itens disponíveis para empacotamento;
- $C$ : capacidade máxima (homogênea) de cada caixa;
- $w_j$ : peso associado ao item  $j$ , tal que  $w_j > 0$ .

#### B. Variáveis de Decisão

- $y_i \in \{0, 1\}$ : variável binária que assume o valor 1 se a caixa  $i$  for utilizada na solução, e 0 caso contrário, para  $i \in \{1, \dots, n\}$ ;
- $x_{ij} \in \{0, 1\}$ : variável binária que assume o valor 1 se o item  $j$  for alocado na caixa  $i$ , e 0 caso contrário, para  $i, j \in \{1, \dots, n\}$ .

#### C. Função Objetivo

A função objetivo do modelo visa minimizar o número total de caixas ativadas para o empacotamento de todos os itens. Ela é expressa matematicamente por:

$$\min \sum_{i=1}^n y_i \quad (1)$$

#### D. Restrições

O modelo matemático está sujeito ao seguinte conjunto de restrições operacionais:

- 1) *Garantia de alocação única*: assegura que cada item  $j$  seja processado e atribuído a exatamente uma única caixa.

$$\sum_{i=1}^n x_{ij} = 1 \quad \forall j \in \{1, \dots, n\} \quad (2)$$

- 2) *Respeito à capacidade limite*: garante que o somatório dos pesos dos itens alocados em uma caixa  $i$  não exceda a sua respectiva capacidade  $C$ . A presença da variável  $y_i$  no lado direito da inequação atua como um fator de ativação (ou restrição lógica): caso a caixa não seja utilizada na solução ( $y_i = 0$ ), a capacidade disponível nela torna-se zero, impedindo a alocação de qualquer item.

$$\sum_{j=1}^n w_j x_{ij} \leq C y_i \quad \forall i \in \{1, \dots, n\} \quad (3)$$

### IV. IMPLEMENTAÇÃO REALIZADA

A implementação computacional do modelo matemático foi desenvolvida integralmente na linguagem Python, utilizando a interface de programação de aplicativos (API) do *solver* comercial Gurobi para a resolução do modelo de Programação Linear Inteira [2]. Conforme as diretrizes metodológicas estabelecidas para este trabalho, o algoritmo implementado corresponde ao *BASIC ILP* (Modelo de Kantorovich). Visando assegurar a organização estrutural, a reprodutibilidade e a escalabilidade dos experimentos, a arquitetura do código-fonte foi segmentada em quatro módulos principais:

- *Leitura e Pré-processamento* (`src/parser.py`): Módulo responsável pela ingestão de dados. Ele realiza a leitura dos arquivos de texto (`.txt`) provenientes do repositório oficial da literatura (BPPLIB), executa a rotina de limpeza de caracteres formatadores e extrai os parâmetros fundamentais da instância: a quantidade total de itens ( $n$ ), a capacidade homogênea das caixas ( $C$ ) e o vetor contendo os pesos individuais de cada item ( $w_j$ ).
- *Modelagem Matemática* (`src/model.py`): Este módulo contém a implementação do modelo *BASIC ILP*, instanciando o ambiente do Gurobi (Optimizer versão 11.0, executado em Python 3.10). A formulação matemática de Kantorovich foi mapeada para os métodos nativos da API (por meio de funções como `addVars`, para a definição das variáveis de decisão booleanas, e `addConstr`, para a inserção algébrica das restrições operacionais). Para garantir a fidelidade metodológica ao experimento do artigo de referência, dois parâmetros de controle foram rigorosamente definidos no *solver*: `TimeLimit = 300`, limitando a execução da árvore de *Branch-and-Bound* a um máximo de 5 minutos; e `MIPGap = 0.0`, configuração que obriga a ferramenta a comprovar matematicamente a otimalidade exata da solução, sem qualquer margem de erro.

- *Execução e Exportação (src/main.py)*: Script principal desenvolvido para automatizar a execução em lote (*batch processing*). O código mapeia um dicionário contendo nove classes distintas de instâncias e processa, de forma iterativa, amostras de cada classe. Ao término da otimização, as métricas de saída — categoria da instância, nome do arquivo, número mínimo de caixas utilizadas e o tempo de processamento de CPU — são estruturadas e exportadas diretamente para um arquivo consolidado em formato tabular (*resultados\_bpp.csv*).
- *Análise de Dados e Visualização (src/analysis.py)*: Módulo suplementar criado para a compilação estatística dos resultados brutos. Ele consolida os tempos de processamento e as taxas de interrupção (*timeouts*), gerando uma tabela de médias (*medias\_bpp.csv*) e plotando automaticamente os gráficos comparativos.

O código-fonte integral, as rotinas de automação e os dados tabulados resultantes desta implementação encontram-se disponíveis publicamente para verificação e reprodutibilidade no repositório associado a este trabalho [3].

## V. AMBIENTE COMPUTACIONAL UTILIZADO

A execução dos experimentos e a validação do modelo matemático foram conduzidas em um ambiente computacional local, cujas especificações de *hardware* e *software* encontram-se detalhadas na Tabela I.

TABLE I  
ESPECIFICAÇÕES DO AMBIENTE COMPUTACIONAL UTILIZADO NOS TESTES LOCAIS.

| Componente                        | Especificação          |
|-----------------------------------|------------------------|
| Máquina                           | Dell Inspiron 13 5330  |
| Processador                       | Intel Core i7-1360P    |
| Memória RAM                       | 16 GB DDR5             |
| Placa de Vídeo                    | Intel Iris Xe Graphics |
| Sistema Operacional               | Zorin OS               |
| Ambiente de Desenvolvimento (IDE) | Antigravity            |

Para fins de consolidação e comparação metodológica, ressalta-se que os autores do artigo de referência executaram seus experimentos originais em um servidor equipado com processador Intel Xeon de 3.10 GHz e 8 GB de memória RAM, restringindo a execução a uma única *thread* e utilizando o *solver* IBM CPLEX na versão 12.6.0 [1]. A evolução do *hardware* atual (processador moderno e maior capacidade de memória DDR5) em relação ao equipamento utilizado no estudo original fornece maior capacidade computacional. Esse fator, somado às atualizações algorítmicas do *solver*, ajuda a justificar eventuais ganhos de performance durante o processo de otimização (*Branch-and-Bound*).

## VI. RESULTADOS OBTIDOS

Para avaliar o comportamento prático do modelo matemático sob diferentes níveis de complexidade, foi conduzida uma bateria de testes computacionais utilizando um recorte de classes da literatura (BPPLIB). O modelo testado

em todos os cenários foi o clássico *BASIC ILP* (Kantorovich), garantindo a fidelidade ao escopo metodológico. O tempo de processamento foi restrito a 300 segundos por instância, com a tolerância de erro (*MIPGap*) ajustada rigorosamente para zero no *solver*. O sumário das métricas de execução extraídas é apresentado na Tabela II.

TABLE II  
RESUMO DO DESEMPENHO COMPUTACIONAL POR CLASSE DE INSTÂNCIA

| Classe da Instância      | Métricas de Execução (Gurobi) |                 |                                 |
|--------------------------|-------------------------------|-----------------|---------------------------------|
|                          | Total Testado                 | Tempo Médio (s) | Status Dominante                |
| Falkenauer U             | 10                            | 20.22           | Ótimo Comprovado                |
| Falkenauer T             | 10                            | 107.54          | Ótimo Comprovado                |
| Scholl 1 (Baixa Cap.)    | 10                            | 0.15            | Ótimo Comprovado                |
| Scholl 2 (Alta Cap.)     | 10                            | 72.81           | Ótimo Comprovado                |
| Scholl 3 (Hard)          | 10                            | 279.08          | Timeout / Subótimo <sup>a</sup> |
| Schwerin 1 ( $n = 100$ ) | 10                            | 65.02           | Ótimo Comprovado                |
| Schwerin 2 ( $n = 120$ ) | 10                            | 125.90          | Ótimo Comprovado                |
| Wäscher                  | 10                            | 224.40          | Timeout / Subótimo <sup>a</sup> |
| Hard28                   | 10                            | 287.00          | Timeout / Subótimo <sup>a</sup> |

<sup>a</sup>Tempo limite de 300s atingido na maioria das amostras da classe.

A análise dos dados evidencia o impacto das características estruturais de cada instância no esforço computacional exigido. A classe *Falkenauer U*, caracterizada por pesos uniformes e capacidade restrita, funcionou como uma linha de base, obtendo comprovação de otimalidade em um tempo médio reduzido. A influência direta da capacidade (*C*) na dificuldade do problema é observada ao se comparar as classes *Scholl 1* e *Scholl 2*: o aumento de *C* elevou o tempo médio de resolução de 0.15 para 72.81 segundos.

A relação do crescimento da complexidade em função do número de itens (*n*) foi validada pelo comparativo entre *Schwerin 1* e *Schwerin 2*, em que o acréscimo de apenas 20 itens resultou quase na duplicação do tempo de processamento. Por fim, conforme ilustrado visualmente na Figura 1, nota-se que as classes rigorosas como *Scholl 3*, *Wäscher* e *Hard28* atingiram o tempo limite estipulado na grande maioria (de 70% a 90%) das amostras, provando a incapacidade do modelo *BASIC ILP* em fechar a árvore de busca dentro da janela de 5 minutos para altas complexidades.

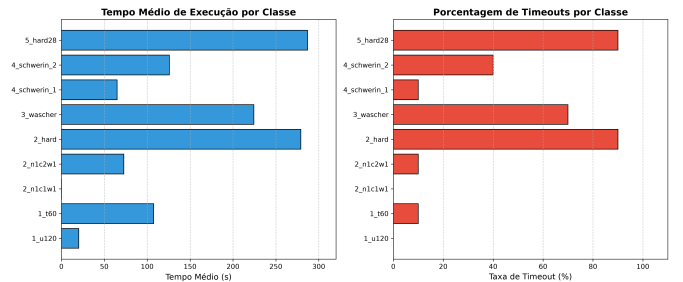


Fig. 1. Comparativo de tempo médio de execução e ocorrência de *timeouts* entre as instâncias testadas.

## VII. COMPARAÇÃO COM OS RESULTADOS DO ARTIGO

Ao confrontar as métricas obtidas localmente com a literatura compilada por Delorme, Iori e Martello (2016) [1], constata-se uma convergência empírica exata em relação às

limitações matemáticas do modelo *BASIC ILP*, mas com avanços notáveis provenientes do hardware moderno. Para formalizar esta análise, a Tabela III contrasta o tempo médio de execução da nossa implementação com o experimento original, o qual operou sob um tempo limite mais restrito de 60 segundos.

TABLE III  
COMPARATIVO EMPÍRICO: NOSSA IMPLEMENTAÇÃO VS. ARTIGO ORIGINAL (MODELO BASIC ILP)

| Classe da Instância | Nosso Tempo Médio (Limite de 300s) | Tempo do Artigo [1] (Limite de 60s) |
|---------------------|------------------------------------|-------------------------------------|
| Scholl 1            | 0.15 s                             | 23.1 s                              |
| Falkenauer U        | 20.2 s                             | 61.4 s                              |
| Scholl 2            | 72.8 s                             | 40.7 s                              |
| Falkenauer T        | 107.5 s (90% Ótimo)                | 60.0 s (100% Timeout)               |
| Scholl 3            | 279.1 s (90% Timeout)              | 60.0 s (100% Timeout)               |
| Wäscher             | 224.4 s (70% Timeout)              | 49.9 s (100% Timeout)               |
| Hard 28             | 287.0 s (90% Timeout)              | 60.0 s (100% Timeout)               |

A análise da Tabela III evidencia de forma objetiva dois fenômenos. O primeiro é o avanço tecnológico: a nossa implementação apresentou tempos significativamente menores em instâncias computacionalmente tratáveis, como *Scholl 1* e *Falkenauer U*. Entretanto, essa diferença não pode ser atribuída exclusivamente ao modelo matemático, pois o ambiente computacional utilizado é substancialmente mais moderno e emprega um *solver* diferente daquele utilizado pelos autores. O avanço mais notável ocorreu na classe **Falkenauer T** (*triplets*): enquanto o artigo relata que o modelo foi incapaz de resolver as instâncias na época devido à forte simetria, nossa execução obteve prova de otimalidade em 90% das amostras em um tempo médio de 107 segundos.

Além dos tempos de processamento, a quantidade mínima de caixas encontrada coincidiu com os valores ótimos reportados pelo artigo para todas as instâncias resolvidas até a prova de otimalidade. Nos casos em que o limite de tempo foi atingido, o *solver* retornou soluções viáveis, porém sem comprovação de otimalidade, fortalecendo a validação prática do modelo.

O segundo fenômeno observado é o limite prático do modelo em instâncias críticas. Nas classes *Scholl 3*, *Wäscher* e *Hard28*, a alta complexidade provocada por capacidades na ordem de milhares e por simetria esgotou o tempo de CPU em ambas as máquinas, batendo nos respectivos tetos estipulados. Isso atesta que, para problemas de alta ordem, a dificuldade está na fraca qualidade da relaxação linear da matriz de Kantorovich. Essa validação empírica corrobora a conclusão do artigo base: para instâncias com estrutura rigorosa e alta capacidade, torna-se mandatória a aplicação de algoritmos exatos mais sofisticados, fundamentados em geração de colunas (*Branch-and-Price*) ou fluxo de arcos.

## VIII. CONCLUSÕES

Este estudo proporcionou uma imersão prática sobre as limitações de modelos puros de programação matemática aplicados na modelagem da Otimização Combinatória. A implementação prática do modelo clássico de Kantorovich via

interface do *solver* comercial Gurobi provou ser tecnicamente efetiva para instâncias favoráveis. Todavia, a barreira de escalabilidade despontou nos testes com as instâncias desafiadoras, como a classe *Wäscher* [1].

Esse mesmo limite metodológico foi reproduzido no ambiente local, no qual tais classes esgotaram o limite de tempo sem a devida prova de otimalidade. Adicionalmente, as dificuldades relatadas no artigo original em lidar com a presença de capacidades elevadas justificam o aumento expressivo no esforço computacional observado na transição de instâncias simples para configurações mais complexas.

Essa validação experimental demonstra na prática a tese central defendida pelos autores: a limitação das formulações compactas clássicas impulsionou o desenvolvimento de algoritmos exatos mais sofisticados, sendo os métodos de geração de colunas e fluxo de arcos os recomendados pela literatura contemporânea para instâncias com estrutura rigorosa.

Dessa forma, o trabalho atingiu seu objetivo de reproduzir uma formulação clássica da literatura, implementar o modelo utilizando o *solver* Gurobi e validar experimentalmente seus resultados frente aos experimentos reportados no artigo de referência.

## REFERENCES

- [1] M. Delorme, M. Iori, and S. Martello, "Bin packing and cutting stock problems: Mathematical models and exact algorithms," *European Journal of Operational Research*, vol. 255, pp. 1–20, 2016.
- [2] Gurobi Optimization, LLC, "Gurobi Optimizer Reference Manual," 2026. [Online]. Available: <https://www.gurobi.com>
- [3] SOUZA, Maria E. T, "Trabalho Final: Otimização Combinatória - 1D Bin Packing Problem (1D-BPP)," GitHub repository, 2026. [Online]. Available: <https://github.com/dudatsouza/bin-packing-optimization>